

Sprint 2: Database Normalization

Smart Parking Reservation System

Team: Shaya Arya, Eduardo Melgoza, Jamil Staten

Overview

Normalization is the process of organizing a relational database to reduce redundancy and ensure data integrity. We apply normalization from First Normal Form (1NF) through Fifth Normal Form (5NF) to each of the six tables in our schema. For each table we first identify all functional dependencies, then verify compliance at each normal form level.

Normal Form Definitions used:

- **1NF:** All attributes are atomic (no repeating groups, no multi-valued columns). Each row is uniquely identifiable.
- **2NF:** In 1NF, and every non-key attribute is fully functionally dependent on the entire primary key (no partial dependencies — only relevant when the PK is composite).
- **3NF:** In 2NF, and no non-key attribute is transitively dependent on the primary key (non-key $\rightarrow$ non-key dependencies are removed).
- **BCNF:** For every non-trivial functional dependency $X \rightarrow Y$, X is a superkey.
- **4NF:** In BCNF, and no multi-valued dependencies exist.
- **5NF:** In 4NF, and no join dependencies exist that are not implied by the candidate keys.

Table 1: USERS

Attributes: user_id, email, password_hash, first_name, last_name, phone, role, is_active, created_at, updated_at

Functional Dependencies

FD1: user_id $\rightarrow$ email, password_hash, first_name, last_name, phone, role, is_active, created_at, updated_at
FD2: email $\rightarrow$ user_id, password_hash, first_name, last_name, phone, role, is_active, created_at, updated_at

user_id and email are both candidate keys. Every other attribute depends directly and solely on the candidate key — there are no partial or transitive dependencies.

Normalization Analysis

1NF: All attributes are atomic. first_name and last_name are stored as separate columns rather than a single full_name field, ensuring atomicity. No repeating groups. Primary key (user_id) uniquely identifies each row. ✓

2NF: The primary key is a single column (user_id), so partial dependency is impossible by definition. All non-key attributes depend on the full key. ✓

3NF: No non-key attribute determines another non-key attribute. role does not determine any other column; phone does not determine name or email. There are no transitive dependencies. ✓

BCNF: Both determinants (user_id and email) are superkeys of the table. Every non-trivial FD has a superkey on the left side. ✓

4NF: No multi-valued dependencies. A user has exactly one email, one role, and one phone number per record. ✓

5NF: No join dependencies beyond those implied by candidate keys. The table cannot be decomposed further without loss. ✓

Result: USERS is in 5NF. No changes required.

Table 2: VEHICLES

Attributes: vehicle_id, user_id, plate_number, make, model, color, year, is_active, created_at

Functional Dependencies

FD1: vehicle_id → user_id, plate_number, make, model, color, year, is_active, created_at

FD2: plate_number → vehicle_id, user_id, make, model, color, year, is_active, created_at

vehicle_id and plate_number are both candidate keys. user_id is a foreign key — it does not determine make, model, color, or year (those are attributes of the vehicle, not the user).

Normalization Analysis

1NF: All attributes are atomic. make, model, color, and year are each single-valued. No repeating groups. ✓

2NF: Single-column primary key. No partial dependencies possible. ✓

3NF: No transitive dependencies. make does not determine model; color does not determine year. user_id is a FK reference, not a transitive path from vehicle attributes to user attributes — user data lives in the USERS table. ✓

BCNF: Both determinants (vehicle_id and plate_number) are superkeys. ✓

4NF: No multi-valued dependencies. A vehicle has exactly one make, one model, one color, and one year. ✓

5NF: No lossless-join decomposition exists. ✓

Result: VEHICLES is in 5NF. No changes required.

Table 3: PARKING_SLOTS

Attributes: slot_id, slot_code, level_name, zone, status, created_at

Functional Dependencies

FD1: slot_id → slot_code, level_name, zone, status, created_at

FD2: slot_code → slot_id, level_name, zone, status, created_at

Considered but rejected: One might argue that $\text{level_name} \rightarrow \text{zone}$ or $\text{zone} \rightarrow \text{level_name}$. In our system, a zone can span multiple levels (Zone A exists on both Level 1 and Level 2), so neither determines the other. No transitive dependency exists.

Normalization Analysis

1NF: All attributes are atomic. Each slot has exactly one code, one level, and one zone. ✓

2NF: Single-column PK. No partial dependencies. ✓

3NF: level_name and zone are independent facts about the slot. Neither determines the other. No transitive dependency. ✓

BCNF: Both candidate keys (slot_id , slot_code) are superkeys on the left side of all FDs. ✓

4NF: No multi-valued dependencies. A slot belongs to one zone and one level. ✓

5NF: No non-trivial join dependencies. ✓

Result: PARKING_SLOTS is in 5NF. No changes required.

Table 4: BOOKINGS

Attributes: booking_id , user_id , vehicle_id , slot_id , start_time , end_time , status , created_at

Functional Dependencies

FD1: $\text{booking_id} \rightarrow \text{user_id}, \text{vehicle_id}, \text{slot_id}, \text{start_time}, \text{end_time}, \text{status}, \text{created_at}$

booking_id is the only candidate key. user_id , vehicle_id , and slot_id are foreign keys — they represent relationships, not functional determinants of booking attributes.

Considered: Does $(\text{slot_id}, \text{start_time}) \rightarrow \text{booking_id}$? Not necessarily — in theory two different bookings could target the same slot at overlapping times (prevented by application logic, not a DB-level unique constraint). So this is not a functional dependency.

Normalization Analysis

1NF: All attributes are atomic. start_time and end_time are single date values, not lists. status is a single string. ✓

2NF: Single-column PK. No partial dependencies. ✓

3NF: No non-key attribute determines another non-key attribute. start_time does not determine end_time (the booking duration is variable). status does not determine user_id . ✓

BCNF: The only non-trivial FD has booking_id (a superkey) on the left. ✓

4NF: No multi-valued dependencies. A booking has one user, one slot, and one vehicle. ✓

5NF: No non-trivial join dependencies. ✓

Result: BOOKINGS is in 5NF. No changes required.

Table 5: PAYMENTS

Attributes: payment_id, booking_id, amount_cents, currency, status, provider, provider_ref, paid_at, created_at

Functional Dependencies

FD1: payment_id → booking_id, amount_cents, currency, status, provider, provider_ref, paid_at, created_at
FD2: booking_id → payment_id, amount_cents, currency, status, provider, provider_ref, paid_at, created_at

payment_id and booking_id are both candidate keys (booking_id has a UNIQUE constraint). Every other attribute is a fact about the payment itself.

Considered: Does provider → currency? No — Stripe supports multiple currencies. Does status → paid_at? No — a paid_at timestamp is not determined by status alone (a failed payment has no paid_at).

Normalization Analysis

1NF: All attributes are atomic. No lists or repeating groups. ✓

2NF: Single-column PK. No partial dependencies. ✓

3NF: No transitive dependencies. provider does not determine amount; currency does not determine status. ✓

BCNF: Both candidate keys (payment_id and booking_id) are superkeys. All FDs have superkeys on the left. ✓

4NF: No multi-valued dependencies. ✓

5NF: No non-trivial join dependencies. ✓

Result: PAYMENTS is in 5NF. No changes required.

Table 6: DISPUTES

Attributes: dispute_id, booking_id, opened_by_user_id, assigned_admin_id, reason, description, status, resolution_note, created_at, resolved_at

Functional Dependencies

FD1: dispute_id → booking_id, opened_by_user_id, assigned_admin_id, reason, description, status, resolution_note, created_at, resolved_at

dispute_id is the only candidate key. booking_id is a FK but not a candidate key here (a booking could theoretically have multiple disputes in a future version, though currently constrained by the application layer).

Considered: Does status → resolved_at? No — resolved_at is nullable and depends on the actual resolution event, not just the status value. Does assigned_admin_id → status? No — one admin can handle disputes in any status.

Normalization Analysis

1NF: All attributes are atomic. description and resolution_note are CLOB (single text values, not lists). reason is a single string. ✓

2NF: Single-column PK. No partial dependencies. ✓

3NF: No transitive dependencies. opened_by_user_id does not determine reason; assigned_admin_id does not determine resolution_note. Each attribute is a direct fact about the dispute. ✓

BCNF: The only non-trivial FD has dispute_id (a superkey) on the left side. ✓

4NF: No multi-valued dependencies. A dispute has one opener, one (optional) admin, and one booking. ✓

5NF: No non-trivial join dependencies. The table cannot be decomposed without loss. ✓

Result: DISPUTES is in 5NF. No changes required.

Summary

Table	1NF	2NF	3NF	BCNF	4NF	5NF
USERS	✓	✓	✓	✓	✓	✓
VEHICLES	✓	✓	✓	✓	✓	✓
PARKING_SLOTS	✓	✓	✓	✓	✓	✓
BOOKINGS	✓	✓	✓	✓	✓	✓
PAYMENTS	✓	✓	✓	✓	✓	✓
DISPUTES	✓	✓	✓	✓	✓	✓

All six tables in the Smart Parking Reservation System satisfy 5NF. No decomposition or restructuring was required because the schema was designed from the ground up with normalization principles in mind: each entity was given its own table, all attributes depend solely on the primary key, and all relationships are expressed through foreign keys rather than repeated data.

The key design decisions that ensured normalization:

- User information is never repeated in vehicles, bookings, or disputes — only user_id is stored as a FK.
- Slot details (level, zone, status) live entirely in parking_slots and are never duplicated in bookings.
- Payment data is separated from booking data into its own table, preventing update anomalies if a payment status changes.
- The two foreign keys on disputes (opener and admin) reference the same users table — this is not a normalization violation, it correctly models two distinct roles.